

Méthode d'analyse des données en grandes dimensions et apprentissage automatique pour la mécanique MAD

CM1 - Introduction à l'apprentissage automatique et réseaux de neurones

Felipe ROCHA

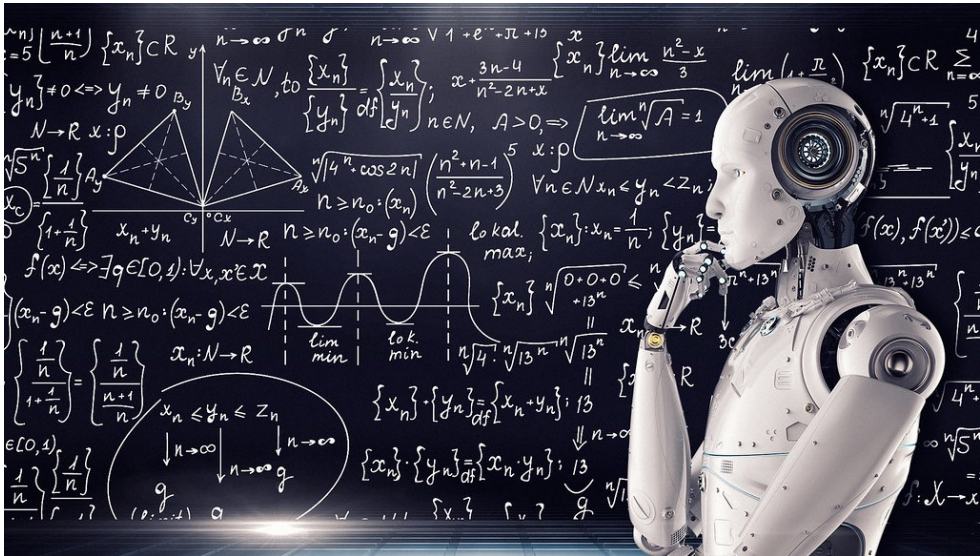
`felipe.figueredo-rocha@u-pec.fr`

dépot git : <https://github.com/felipefr/mad/> (CC BY-NC-SA 4.0)

Faculté des Sciences et Technologie, Univ. Paris-Est Créteil Val de Marne

(16/09/26)

Intelligence Artificiel

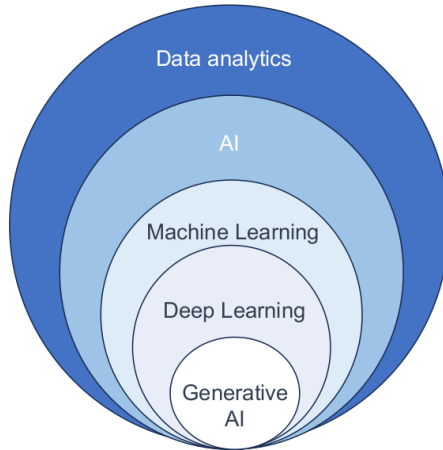


Structure du Cours

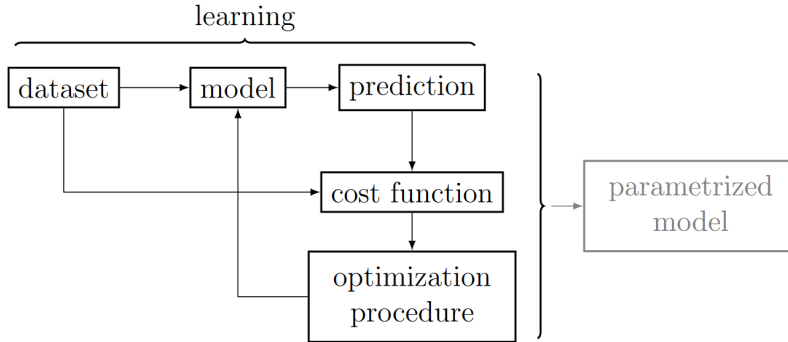
- 4 CMs : 16/09, 23/09, 07/10, 17/11
- 3 TPs (UPEC) + 1 TP (UGE) : grM : 30/09, 14/10, 09/12, grC/FA : 07/10, 14/10, 09/12
- Note 1 : Soutenance d'un mini-projet et TPs (50% de la note finale).
- Note 2 : Examen final écrit (50% de la note finale) portant sur les fondements théoriques (CMs) et abordés lors de TPs.
- Rendus : En forme de slides (qui feront partie de votre soutenance). Deux dates: i) 13/11 et (2 jours avant la soutenance, à programmer).
- Optionnel : Questions théoriques de fin de CMs.

- $\underline{a}$: Vecteur.
- $\underline{\underline{A}}$: Matrice ou Tenseur de 2^{ème}-ordre.
- $[\underline{a} \otimes \underline{b}]_{ij} = a_i b_j$: produit tensoriel
- $\underline{a} \cdot \underline{b} = \sum_i a_i b_i$: produit scalaire
- $[\underline{a} \odot \underline{b}]_i = a_i b_i$: produit composante par composante
- $\underline{b} = \underline{\underline{A}} \underline{x} \Leftrightarrow b_i = A_{ij} x_j \Leftrightarrow \underline{b} = A_{ij} x_j \underline{e}_i$: Notation d'Einstein.
- $\delta_{ij} = \begin{cases} 1, & i = j \\ 0, & \text{cc} \end{cases}$: Delta de Kronecker.
- Exemple : $\frac{d}{dx_k}(A_{ij} x_j) = A_{ij} \delta_{jk} = A_{ik}$.

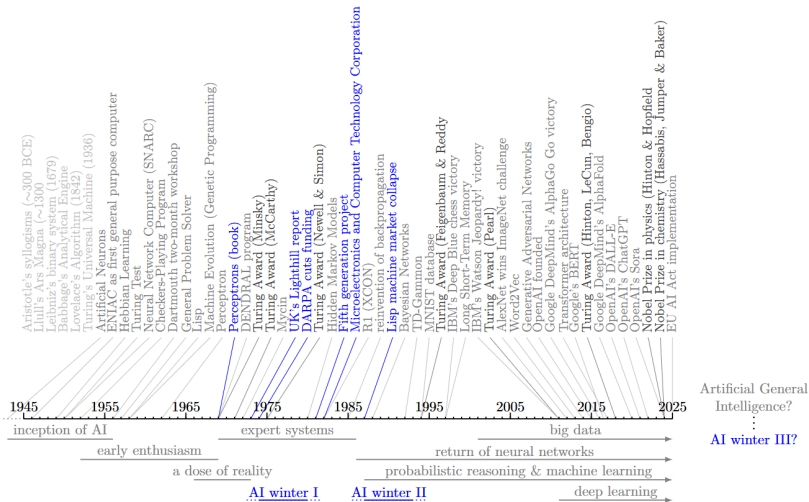
Intelligence Artificiel vs Apprentissage Automatique



Apprentissage Automatique (Machine Learning)

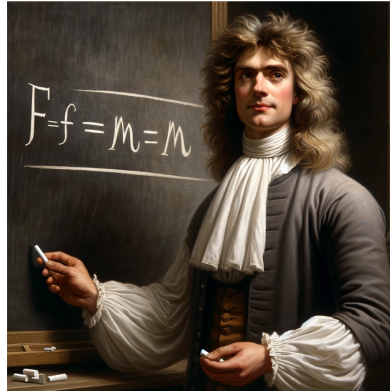
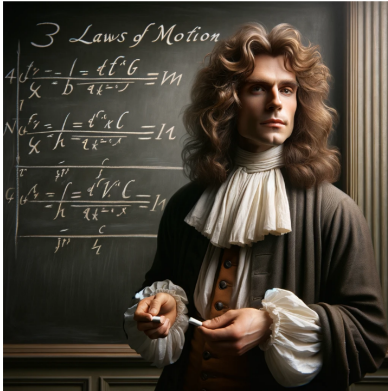


Histoire de l'IA

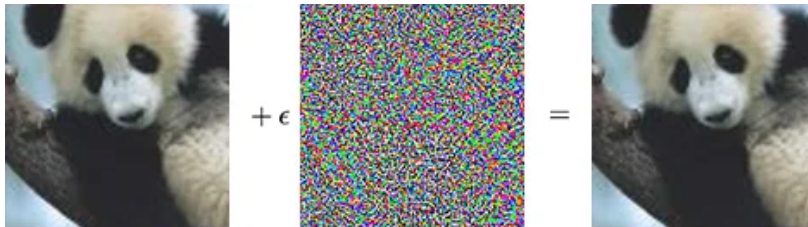


Challenges

- Prompt 1: "Generate an image of Isaac Newton in front of a blackboard on which his three laws are written in mathematical notation and chalk."
- Prompt 2: "The laws on the blackboard are incorrect. Please add the correct formulations. If you are unable to do so, simply focus on the second law, which is $F=m \cdot a$."



Challenges



"panda"

57.7% confidence

"gibbon"

99.3% confidence

- Les réseaux de neurones peuvent échouer de manière imprévisible → ils peuvent être systématiquement trompés.
- Le deep learning n'est pas robuste en raison de sa sensibilité aux hyperparamètres → il nécessite un réglage approfondi.
- Les réseaux de neurones sont difficilement interprétables, c'est-à-dire qu'ils offrent une explicabilité limitée → cela limite leur fiabilité.

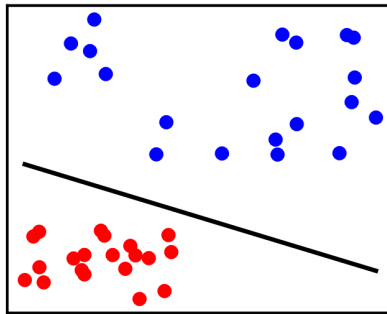
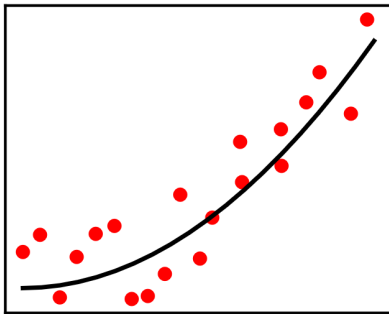
Classification algorithmes d'apprentissage

Concernant le type d'apprentissage :

- Supervisé
- Non-supervisé
- Semi-supervisé
- Apprentissage par renforcement

Vis-à-vis la tâche

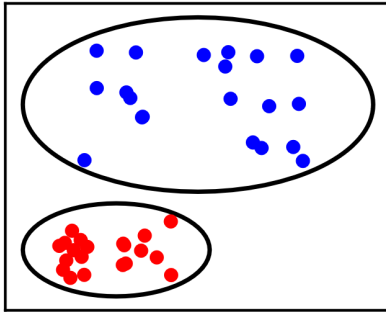
- Classification
- Regression
- Clustering
- Génération



Classification algorithmes d'apprentissage

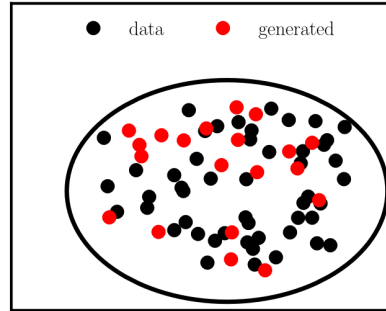
Concernant le type d'apprentissage :

- Supervisé
- Non-supervisé
- Semi-supervisé
- Apprentissage par renforcement



Vis-à-vis la tâche

- Classification
- Regression
- Clustering
- Génération

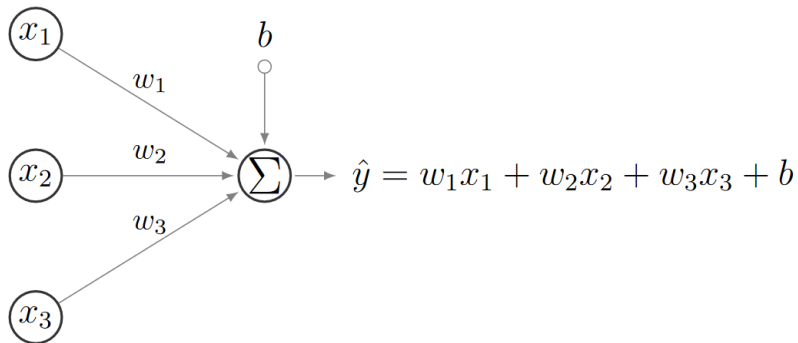


Méthodes d'apprentissage/IA

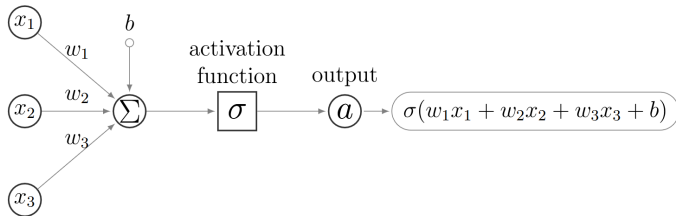
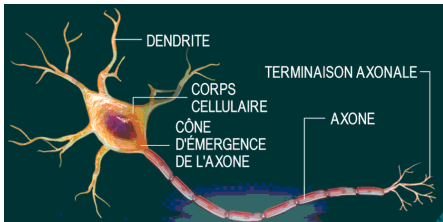
Classifications	Algorithms	Tasks
Supervised	Decision trees	Classification
	Random forest	Classification, regression
	Support vector machines	Classification, regression
	K-nearest neighbours	Classification
	Bayesian network	Classification
	Gaussian process	Regression
	Multi-gene genetic programming	Regression
	Hidden semi-Markov model	Classification
	Multi-layer perceptron	Classification, regression
	Convolutional neural network	Classification
	Recurrent neural network	Time series prediction (regression)
	Adaptive network-based fuzzy inference system	Regression
	Transformers	Regression, classification, generative modeling
Unsupervised	Self-organizing map	Clustering
	Deep belief network	Classification
	K-means clustering	Clustering
	Reduced order modeling (POD)	Dimensionality reduction
	Autoencoder	Generative modeling, dimensionality reduction
	Generative adversarial networks	Generative modeling, (classification)
Semi-supervised	Diffusion model	Generative modeling
	Gaussian mixture model	Clustering

Regression linéaire

Exemple : Trouver $f : \mathbb{R}^3 \rightarrow \mathbb{R}$ de la forme $f(\underline{x}) = \underline{w} \cdot \underline{x} + b$ tel que minimise l'écart moyen carré par rapport à la base de données $D = \{(\underline{x}_i, y_i) \in \mathbb{R}^3 \times \mathbb{R}; i = 1, \dots, N_D\}$.



Perceptron de Rosenblatt



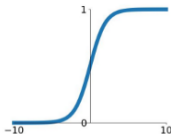
Modèle mathématique :

$$z = \underline{w} \cdot \underline{x} + b, \quad (1)$$

$$\sigma(z) = \begin{cases} 1, & \text{si } z \geq 0, \\ 0 & \text{cc} \end{cases} \quad (2)$$

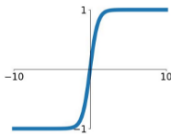
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



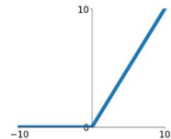
tanh

$$\tanh(x)$$



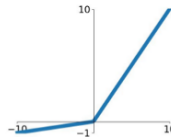
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

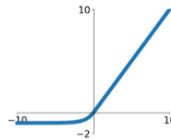


Maxout

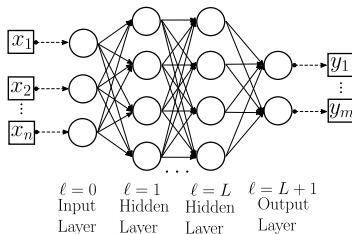
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Réseaux de Neurones : Architecture Multicouches dense



- Base de données : $D = \{(\underline{x}_i, \underline{y}_i) \in \mathbb{R}^n \times \mathbb{R}^m; i = 1, \dots, N_D\}$
- Réseaux de Neurones: $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$
- Nombre de couches cachées et neurones par couche: L et $d_1, d_2, \dots, d_L$ ($d_0 = n, d_{L+1} = m$)
- Signaux rentrant dans la couche $\ell \in \{1, \dots, L\}$: $\underline{z}^{(\ell)} = \underline{\underline{W}}^{(\ell)} \underline{a}^{(\ell-1)} + \underline{b}^{(\ell)}$
- Poids et *Bias* de la couche $\ell \in \{1, \dots, L + 1\}$: $\underline{\underline{W}}^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, \underline{b}^{(\ell)} \in \mathbb{R}^{d_\ell}$ (resp.)
- Signaux et fonctions d'activation de la couche ℓ (supposé homogène) : $\underline{a}^{(\ell)} = \underline{\sigma}^{(\ell)}(\underline{z}^{(\ell)})$
- Cas particuliers : $\underline{a}^{(0)} = \underline{x}$, $\underline{a}^{(L+1)} = \underline{\hat{y}} = \underline{z}^{(L+1)}$ (supposant identité dans la fonction d'activation de la sortie).

- Paramètres d'entraînement:

$$\underline{\Theta} = (\underline{W}^{(1)}, \underline{b}^{(1)}, \dots, \underline{W}^{(L+1)}, \underline{b}^{(L+1)}) \quad (3)$$

- Fonction Coût (Loss Function):

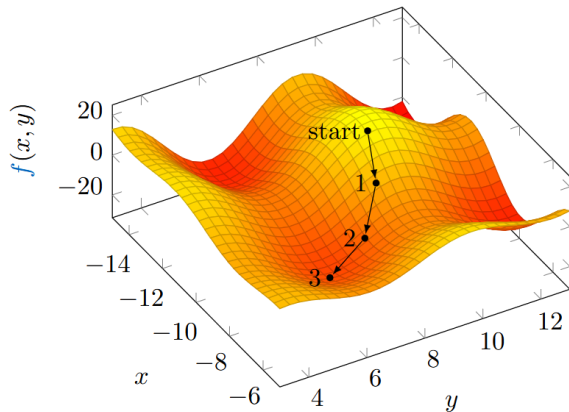
$$L(\underline{\Theta}) = \frac{1}{N_D} \sum_{i=1}^{N_D} \|\hat{y}_i - y_i\|^2 = \frac{1}{N_D} \sum_{i=1}^{N_D} \|f_{\underline{\Theta}}(\underline{x}_i) - y_i\|^2 \quad (4)$$

- Descente de gradient:

$$\underline{\Theta}_{k+1} = \underline{\Theta}_k - \eta \nabla_{\underline{\Theta}} L \quad (5)$$

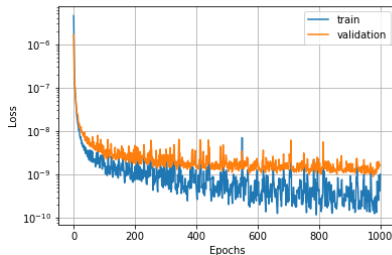
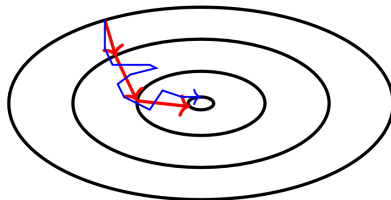
- Descente de gradient stochastique : gradient calculé avec un jeu de données
- Descente de gradient stochastique version mini-batch: gradient calculé avec B jeux de données
- Epochs : nombre de pas (si stochastique, il est nécessaire de parcourir complètement la base de donnée).

Visualisation entraînement



stochastic
gradient descent

full-batch
gradient descent



Algorithme Backpropagation

- Propagation forward:

$$\underline{x} \rightarrow \underline{z}^{(1)} \rightarrow \underline{a}^{(1)} \rightarrow \underline{z}^{(2)} \rightarrow \underline{a}^{(2)} \rightarrow \dots \rightarrow \underline{z}^{(L)} \rightarrow \underline{a}^{(L)} \rightarrow \underline{z}^{(L+1)} \rightarrow \underline{\hat{y}} \quad (6)$$

- Propagation backward ($B = 1$):

$$\underline{\delta}^{(L+1)} = (\underline{\hat{y}} - \underline{y}) \odot (\underline{\sigma}^{(L+1)})'(\underline{z}^{(L+1)}), \quad (7)$$

$$\downarrow$$
$$\underline{\delta}^{(\ell)} = ([\underline{\underline{W}}^{(\ell+1)}]^T \underline{\delta}^{(\ell+1)}) \odot (\underline{\sigma}^{(\ell)})'(\underline{z}^{(\ell)}), \quad (8)$$

$$\downarrow$$
$$\underline{\delta}^{(1)} = ([\underline{\underline{W}}^{(2)}]^T \underline{\delta}^{(2)}) \odot (\underline{\sigma}^{(1)})'(\underline{z}^{(1)}) \quad (9)$$

- Gradients:

$$\nabla_{\underline{b}^{(\ell)}} L = \underline{\delta}^{(\ell)} \quad , \quad \nabla_{\underline{\underline{W}}^{(\ell)}} L = \underline{\delta}^{(\ell)} \otimes \underline{a}^{(\ell-1)} \quad (10)$$

Algorithme Backpropagation (Mini-Batch)

- $\underline{\underline{X}} \in \mathbb{R}^{n \times B}, \underline{\underline{Y}} \in \mathbb{R}^{m \times B}$

- Propagation forward:

$$\underline{\underline{X}} \rightarrow \underline{\underline{Z}}^{(1)} \rightarrow \underline{\underline{A}}^{(1)} \rightarrow \dots \rightarrow \underline{\underline{Z}}^{(L)} \rightarrow \underline{\underline{A}}^{(L)} \rightarrow \underline{\underline{A}}^{(L+1)} \rightarrow \underline{\underline{\hat{Y}}} \quad (11)$$

- Propagation backward:

$$\underline{\underline{\Delta}}^{(L+1)} = (\underline{\underline{\hat{Y}}} - \underline{\underline{Y}}) \odot (\underline{\underline{\sigma}}^{(L+1)})'(\underline{\underline{Z}}^{(L+1)}), \quad (12)$$

$\downarrow$

$$\underline{\underline{\Delta}}^{(\ell)} = ([\underline{\underline{W}}^{(\ell+1)}]^T \underline{\underline{\Delta}}^{(\ell+1)}) \odot (\underline{\underline{\sigma}}^{(\ell)})'(\underline{\underline{Z}}^{(\ell)}), \quad (13)$$

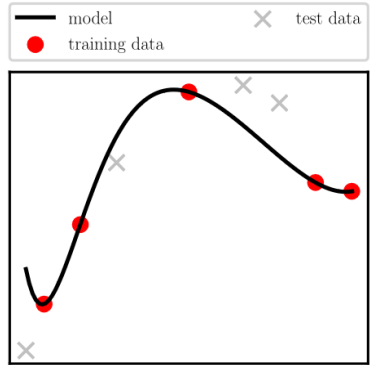
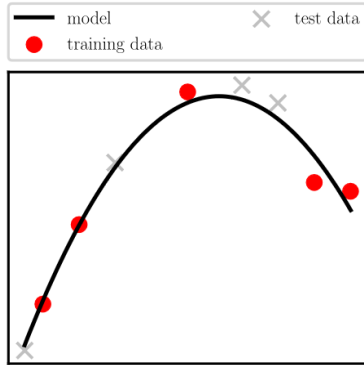
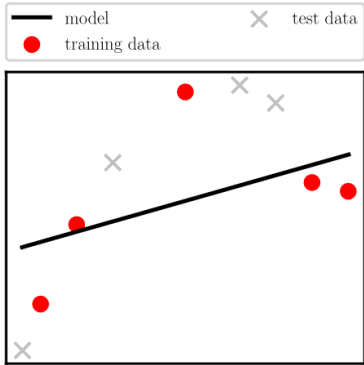
$\downarrow$

$$\underline{\underline{\Delta}}^{(1)} = ([\underline{\underline{W}}^{(2)}]^T \underline{\underline{\Delta}}^{(2)}) \odot (\underline{\underline{\sigma}}^{(1)})'(\underline{\underline{Z}}^{(1)}) \quad (14)$$

- Gradients:

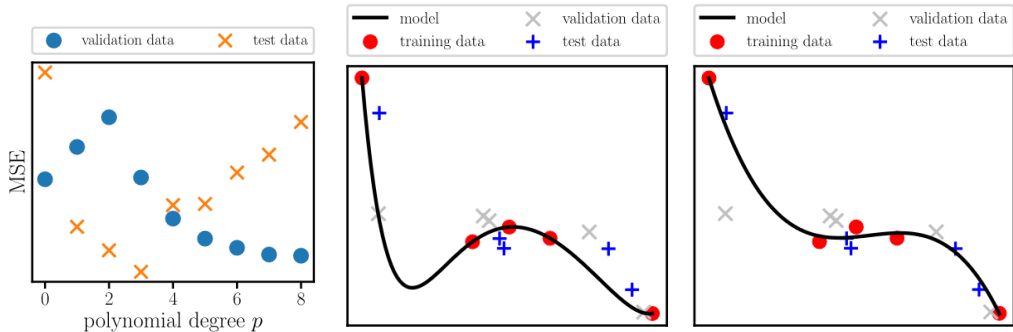
$$\nabla_{\underline{\underline{b}}^{(\ell)}} L = \underline{\underline{\Delta}}^{(\ell)} \underline{\underline{1}}_B, \quad \nabla_{\underline{\underline{W}}^{(\ell)}} L = (\underline{\underline{\Delta}}^{(\ell)} \underline{\underline{1}}_B) \otimes (\underline{\underline{A}}^{(\ell-1)} \underline{\underline{1}}_B) \quad (15)$$

Surapprentissage (Overfitting)

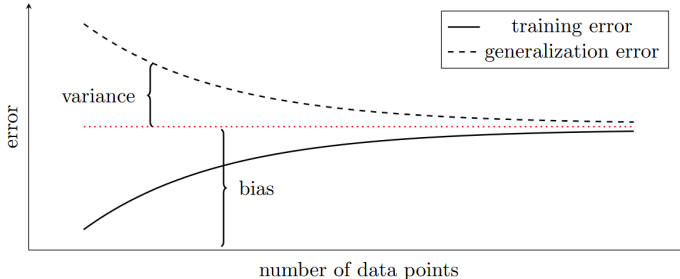
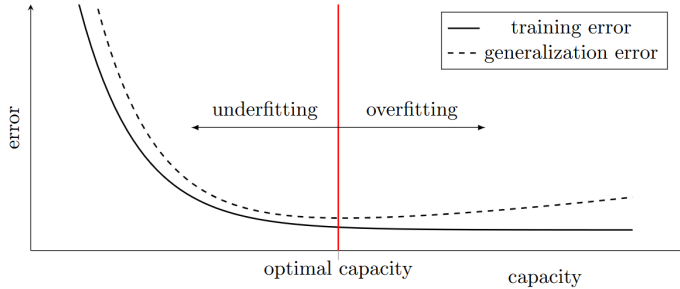


Surapprentissage (Overfitting)

- Division des jeux données : entraînement, validation, test (e.g 80/20/20, 90/10/10, etc)

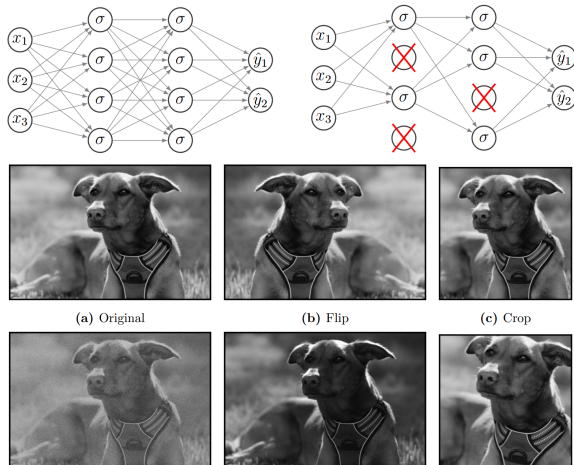


Surapprentissage (Overfitting)



D'autres détails techniques

- Prevention overfitting (sur-apprentissage):
 - regularisation L2 ou L1
 - early-stopping
 - dropout
- Décroissance programmé du Learning-rate
- Mise à l'échelle des entrées et de la sortie : MinMaxScaler, Normalisation, ...
- Augmentation des données



D'autres méthodes d'optimisation : ADAM

- **ADAM** (*Adaptive Moment Estimation*)
- Il combine :
 - la moyenne des gradients (momentum) ;
 - la moyenne des gradients au carré (adaptation du taux d'apprentissage).
- Le taux d'apprentissage est ainsi **adapté automatiquement** pour chaque paramètre.
- **Avantages** : convergence rapide et peu de réglage manuel.
- **Formulation** : Soit $\underline{g}_t = \nabla_{\underline{\Theta}} \mathcal{L}(\underline{\Theta}_t)$ le gradient à l'itération t .

$$\underline{m}_t = \beta_1 \underline{m}_{t-1} + (1 - \beta_1) \underline{g}_t \quad (16)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (17)$$

Correction du biais : $\hat{\underline{m}}_t = \frac{\underline{m}_t}{1 - \beta_1^t}$, $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$. Mise à jour des paramètres :

$$\boxed{\underline{\Theta}_{t+1} = \underline{\Theta}_t - \eta \frac{\hat{\underline{m}}_t}{\sqrt{\hat{v}_t} + \varepsilon}} \quad (18)$$